A GENERAL-PURPOSE RUNTIME SOFTWARE FACTORY

# Endoforge

## *The entire idea*

**Software creation as a runtime operation.**

Broad intake. Local production.

Independent qualification.

Live use. Selective refinement.

A factory that accumulates capability.

The full concept, from the first RAM-resident workcell to measured self-improvement and the limits of necessary computation.

Concept synthesis · Edition 2
16 October 2026

# The complete story

Read this document for the whole idea. Use the preserved engineering library for subsystem details, original evidence, and working templates.

1. Software becomes something the runtime can make

2. One system, not a bundle of adjacent features

3. The factory is the production machinery

4. The broad funnel: admit quickly, refine selectively

5. One request through the entire factory

6. Improvement is a controlled production process

7. The next frontier is removing repeated understanding

8. The memory and trust architecture must support the idea

9. Retained software is the compounding asset

10. The ledger belongs inside the factory, not at its center

11. Reason upward from physics, not downward from today's workflow

12. Unified-memory systems are a physical fit, not an automatic win

13. The applications are expressions of the same substrate

14. What is supported now, and what remains to be demonstrated

15. From today's components to the possible

Appendix A. Reading the supporting studies as parts of one system

Appendix B. Source and status key

## The idea in one passage

Endoforge makes software creation an operation that a running system can perform, rather than a separate project that must happen before the system can do something new. A person or another program brings a need. The runtime finds existing capabilities, accepts useful code in an appropriate supported form, or generates what is missing. It routes that material into a common WebAssembly execution system, checks it, runs it against the task, measures it, and supplies a working capability. The repeated computational loop stays inside a RAM-resident workshop isolate. The machinery stays prepared; the individual workpieces remain private, bounded, and replaceable. [C; W, pp. 1–7]

The output can be a function, tool, transformation, parser, workflow, simulation component, or a small application with a working interface. Once the behavior exists as code, ordinary interactions execute that code instead of spending another model turn recreating the reasoning. Useful results can be retained. Promising alternatives can be compared. Keepers can earn a careful Rust rewrite, a better algorithm, a smaller dependency closure, or a more specialized execution path. The identity of the capability survives even when its implementation changes. [C; W, pp. 8–10, 23; A, sheets 12–14, 29]

The factory can then apply the same disciplined process to its own machinery. It can improve compilation, memory handling, scheduling, candidate selection, evaluation, and reuse without allowing a candidate to redefine correctness or approve itself. Over time the objective is not to generate more source code. It is to require less new work to deliver the same accepted result, and to make genuinely new capabilities economical at the moment they are needed. [C; W, pp. 9–11]

**The complete idea is the combination: broad intake, local production, independent qualification, live use, selective refinement, accumulated capability, and improvement of the factory itself.** Field OS is an application and early evidence source. TypeScript is an initial route. Mononoke is a source of ledger mechanisms. Unified-memory hardware is a possible physical foundation. None of those is the definition of Endoforge.

## How this document is grounded

This is a cohesive account of the idea developed in the conversation, not a new deployment report or a claim that all proposed subsystems already exist. **C** denotes the conversation and the user's selected direction. **W** is the original 25-page *In-Isolate Software Workshop* white paper. **A** is the original 30-sheet diagram atlas. **M** is the commit-pinned Mononoke study. **R** refers to the previously assembled engineering reference, preserved in full in this archive. References to external work are carried forward from that reference library; they have not been freshly reverified for this synthesis.

The source documents begin with a TypeScript-centered workshop. The conversation deliberately broadens it into a language-agnostic funnel and a real runtime manufacturing system. This document preserves that development instead of silently treating the original implementation as if it already embodied the larger design. Proposals below are proposals unless a passage explicitly identifies a recorded result.

# 1. Software becomes something the runtime can make

The starting point is a change in the relationship between an application and its user. Instead of choosing among only the behaviors that developers anticipated, the user can request an additional capability while working. The application does not merely explain how somebody might implement it. It can manufacture, qualify, and supply the necessary software through Endoforge. [C; W, pp. 2, 23]

The distinction matters even for a small request. A comparison panel may need custom grouping, unit handling, validation, source inspection, and an export rule. Those requirements can be too specific for a generic interface, yet too small to justify a conventional application project. Endoforge's proposition is to reduce the marginal cost of turning that need into working behavior. Whether it succeeds is an economic and user-experience question as well as an execution-speed question. [W, pp. 2, 8, 22]

A successful generated interface is not a screenshot or a decorative layer over another conversation. It contains real behavior: state transitions, calculations, input checks, filtering, error handling, and approved actions. The surrounding application still provides identity, rendering, accessible primitives, source authority, and effect approval. The user sees an instrument suited to the task, not the factory's internal search. [W, p. 8; A, sheets 12–14]

The same production system need not produce a UI at all. Another agent could request a parser. A workflow could need a transformation. A simulation could need a scoring function. An application could request a new interaction handler. These are different products of the same runtime machinery. This is why the platform is broader than agentic coding and broader than generative UI, even though both are important uses. [C; W, pp. 2, 23]

## The cost being challenged

The relevant cost is not just the final function's execution. It is the complete cost of obtaining the function: reasoning, preparation, imports, source handling, checking, compilation, testing, measurement, repair, publication, and subsequent operation. In the proposed factory, reusable machinery and locality remove avoidable preparation and transport; retained software avoids repeating useful reasoning. Neither mechanism makes necessary computation free. [W, pp. 7, 11]

The north-star unit is therefore **time and total resources per independently accepted useful capability**. For improvement campaigns it becomes accepted improvement under an equal complete budget. Tokens, commits, compiler checks, and agent counts can explain a result, but they are not the product objective.

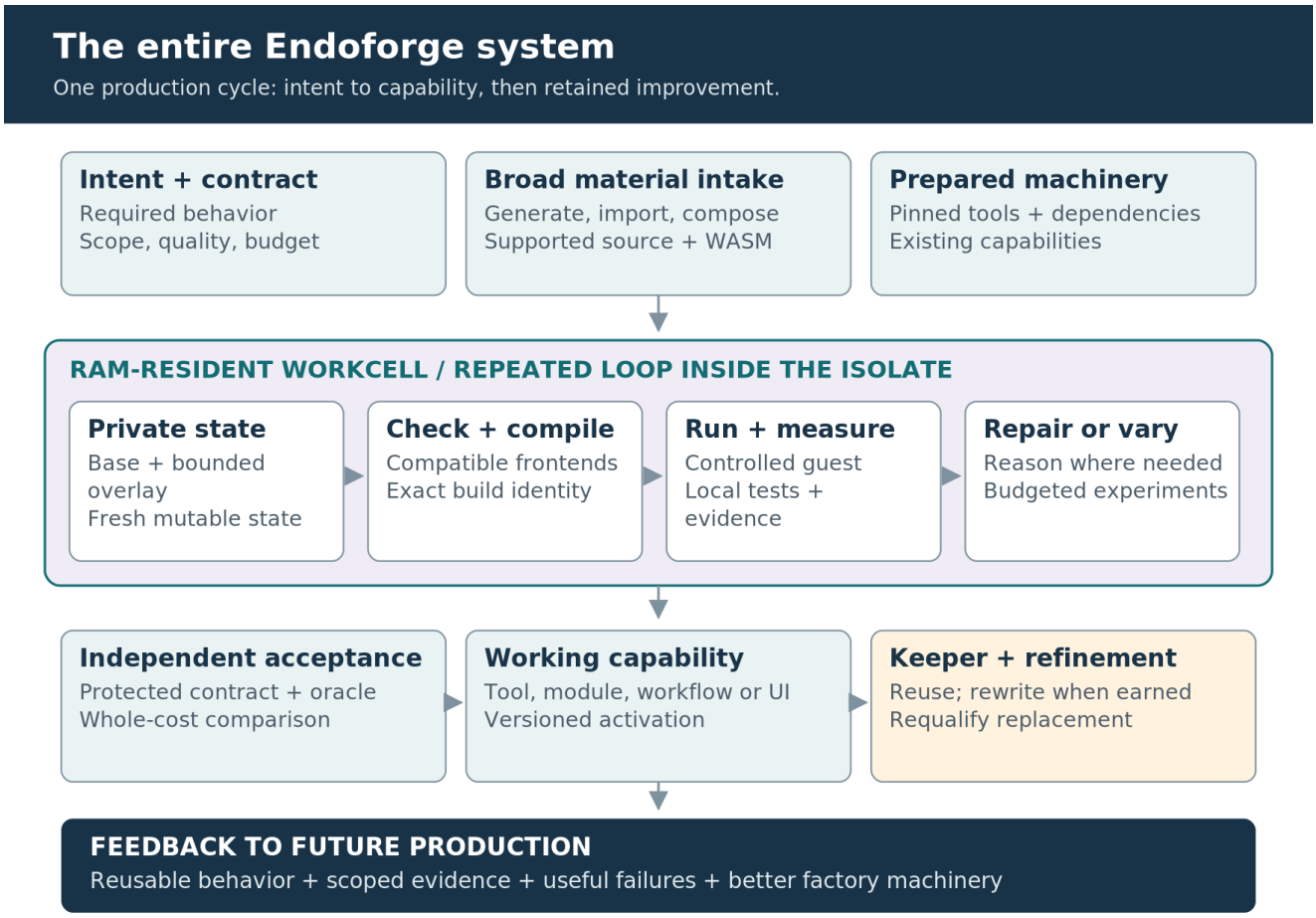
## What should feel different

A person should be able to request a new capability, inspect what it does, use it repeatedly, change the requirement, and keep a useful version. A revision should not destroy the state of the current application. A retained capability should not depend on an indefinitely live guest heap or on repeating the original model conversation. A failed experiment should not replace the working version. [W, pp. 6, 8, 21–23]

The ambition is not constant visible mutation. It is a stable application experience backed by a production process capable of supplying new behavior when needed.

# 2. One system, not a bundle of adjacent features

Endoforge can be understood as a closed production cycle with explicit boundaries. Intent and admitted material enter. A bounded workcell turns them into executable candidates. Independent evidence determines what is eligible for use. An accepted capability serves the task. Its implementation, evidence, and useful structure become material for future work. [C; A, sheets 01, 06, 29]




**Concept map 1. The whole Endoforge system: admitted intent and source enter a local workcell; evidence controls activation; retained capabilities and factory improvements return to future production.**

## Intake and contract

The first boundary establishes what is being asked, which inputs are authoritative, what behavior is required, what resources may be spent, and which effects are permitted. Imported code is source material, not authority. A task contract may need clarification or a new observation before any amount of synthesis can answer it correctly. [C; W, pp. 10, 20]

## The workcell

The repeated path contains the private workspace, compatible compiler or runtime adapters, approved dependencies, candidate execution, local tests, and instrumentation. Its defining direction is

in-isolate and in RAM. Prepared tooling should not be rebuilt or transported through another environment for each small change. Local structure is the center of the design, not a minor optimization applied after a distributed coding workflow has been assembled. [C; W, pp. 4–7]

### Qualification and activation

Local repair feedback and final acceptance serve different purposes. The candidate can receive diagnostic information and propose another implementation. It cannot edit protected tests, choose its own trusted measurement boundaries, widen its permissions, or certify itself. Activation identifies the exact artifact and changes the active version under a controlled revision. [W, pp. 9–10, 20–21]

### The compounding loop

Successful work produces more than one output. It can leave a useful capability, a reusable component, a validated transformation, a counterexample, a more effective search recipe, or an improvement to the factory. These assets should reduce future work within their actual scope. A capability library is not a license to reuse customer-specific data without permission, and a measurement from one environment is not a universal performance certificate. [C; A, sheets 20, 29]

This is the system-level connection among the topics discussed. The compiler funnel creates alternatives. The ledger gives them identity without a workspace per candidate. The evaluator turns alternatives into evidence. The runtime supplies stable use. Keeper refinement improves lifetime economics. The library and search history make later requests less expensive. The factory's own optimization loop improves the capacity to repeat the process.

## 3. The factory is the production machinery

The user's factory framing is a concrete design direction: Endoforge should embody the logic of a working production system, not merely assign manufacturing titles to coding agents. The distinction is between automating people moving among tools and engineering the tools, routing, inventory, inspection, and flow as a system. [C]

The workpiece is an exact candidate implementation or configuration against a task contract. The machinery is the reusable compiler, runtime, dependency base, evaluator, and prepared execution interface. Material includes admitted source, inputs, reusable capabilities, and existing evidence. A manufacturing cell is a bounded active workshop. Finishing is deeper engineering of a capability that has justified the investment.

### Setup and changeover

An ordinary candidate change should not imply installing another compiler, reconstructing another dependency tree, or reconnecting a complete development environment. The factory prepares compatible tooling and approved material once within a measured residency policy. Changeover means changing the candidate and resetting its mutable state, not automatically recreating all surrounding machinery. [W, pp. 6–7]

A rarely used language route need not remain resident forever. Prepared machinery can be loaded on demand, and warm caches can be evicted. Losing an optional cache should increase the cost of the

next attempt, not change correctness or erase the task. This is how local production avoids becoming unbounded inventory. [W, p. 6; A, sheet 07]

## Flow and work in process

A thousand hypotheses do not require a thousand live compiler heaps. Candidate identities, source deltas, priorities, and continuations can wait as records. Only the measured number of active workcells consumes execution resources. Waiting on a user, model, network response, or review should release the slot when a resumable checkpoint is available. [A, sheets 10, 23]

The scheduler should keep the constrained stage supplied without flooding it. Faster proposal generation can be counterproductive when evaluation is already saturated. More compilation can create only a longer queue if the next stage cannot accept the output. Endoforge therefore measures the whole flow and bounds active work rather than treating maximum concurrency as success. [C; W, pp. 11, 21]

## Inspection, rework, and finishing

Cheap deterministic checks reject malformed or incompatible candidates early. More expensive behavioral, integration, UI, and performance evaluation is spent on survivors. Repair feedback should identify the actual failure. The accepted incumbent remains in service while an alternative is manufactured and qualified. [A, sheets 10–14]

A keeper may receive a different process route from a disposable candidate. Once the behavior is worth retaining, more careful engineering can reduce runtime cost or improve maintainability. This does not mean the first stage is allowed to be incorrect. It means the first stage optimizes economical discovery, while the finishing stage optimizes lifetime operation under the same required behavior. [C]

## Scrap and reclamation

Discarding a name is not the same as freeing the underlying storage. The Mononoke source study provides a specific warning: its test-only memory store removes link entries without reclaiming payload entries on those paths. That is not evidence about production-store leakage; it is a reason to design Endoforge's long-lived memory ownership explicitly. [M, section 5]

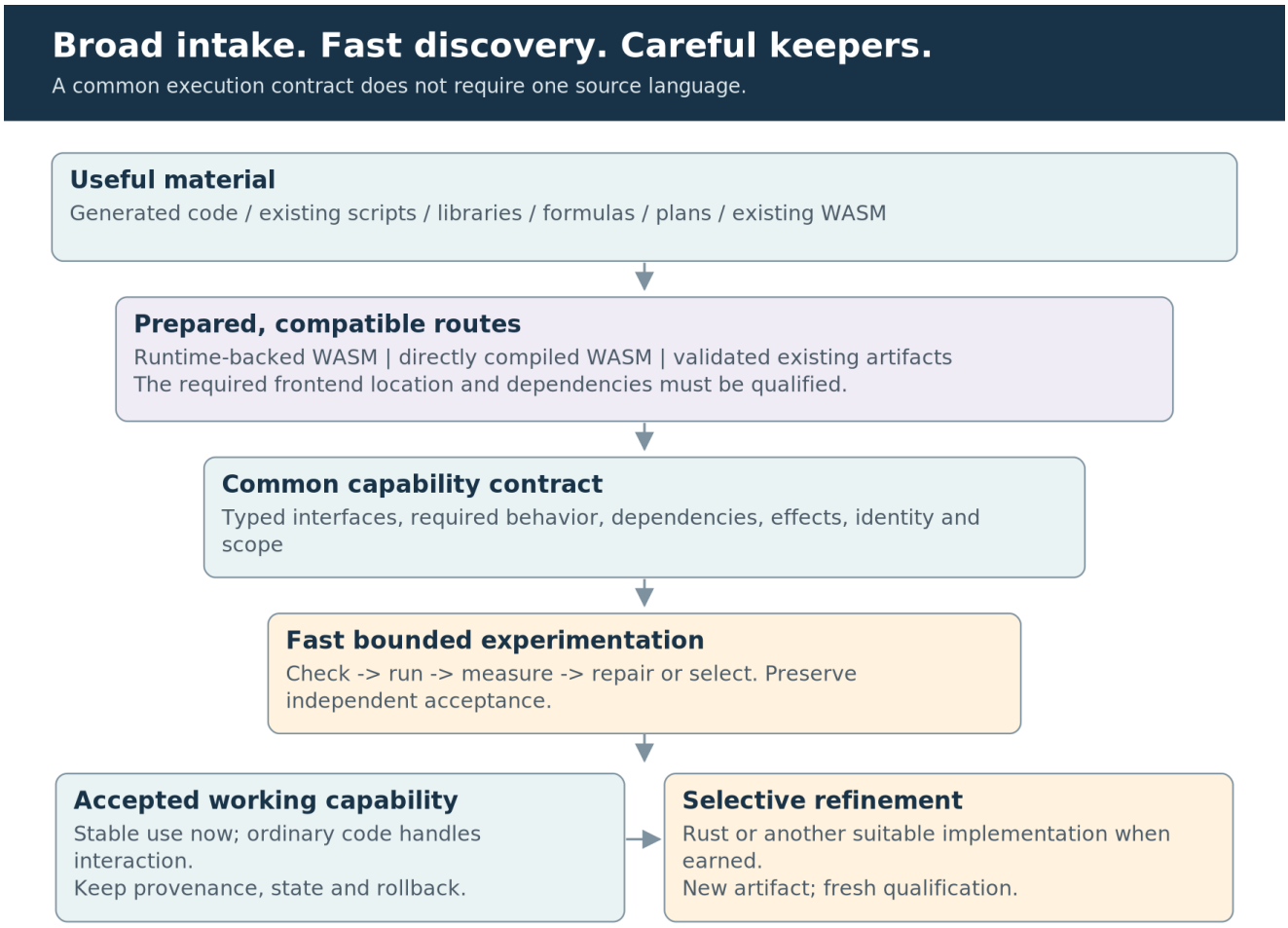
After a campaign, the system needs to know what remains live, what has been retained deliberately, what is awaiting persistence, and what can actually be reclaimed. The factory measures bytes, not just candidate counts. Shared immutable roots and private workpieces need compatible reclamation rules.

## Throughput with quality

The goal is more accepted useful output per unit of time and total resources, not higher utilization at every station. A useful scorecard contains first-pass yield, rework cost, time to evidence, accepted output, memory residency, tail latency, and the lifetime return from retained improvements. These are proposed operating measurements for Endoforge, not existing production results. [C; W, pp. 10–11]

# 4. The broad funnel: admit quickly, refine selectively

The broad funnel is central to the user's idea. Endoforge should generate or ingest code in whatever supported form is appropriate to the task, get it into the common WASM execution system as quickly as practical, evaluate many candidates, and devote careful implementation effort to the keepers that deserve it. Rust is an optional refinement route, not the price of admission. [C]




**Concept map 2. The broad intake funnel, followed by bounded exploration, independent selection, stable capability use, and selective keeper refinement.**

## There are different ways to reach WASM

A directly compiled program and a language runtime packaged in WASM are different execution mechanisms. Both can serve the funnel when their compatibility and costs are explicit. A runtime-backed candidate can be a fast way to test imported behavior without requiring an immediate rewrite. A directly compiled implementation can be attractive when execution dominates or when the capability becomes frequently reused. [C; R03]

The conversation examined AssemblyScript as a TypeScript-like direct-WASM route, JavaScript through runtime-backed tools such as Javy, Python through compatible component packaging, existing Rust or other WASM-targeting source, and already-built WASM. These are candidate adapter

families from the prior discussion, not a verified Endoforge support matrix. Producing WASM externally does not establish that the required compiler runs inside the workcell. [R03; R18, compiler reference group]

For the strict factory path, the required frontend, shims, admitted dependencies, and evaluator must actually operate inside the chosen isolate and resource envelope. Initial preparation of the factory's tooling is separate from repeatedly compiling its workpieces. A route that still requires an external build for each candidate remains an explicit compatibility lane or an open integration task; it is not silently counted as satisfying the all-local objective.

## A compiler written in Rust is not the same as compiling the candidate to Rust

The original workshop has two distinct transformations. Rust source is used to build the WASM TypeScript checker/emitter. Generated TypeScript is checked and emitted as JavaScript. The candidate does not become Rust or direct-WASM computation merely because its compiler implementation is Rust. The later funnel broadens this model rather than changing what the original route demonstrably does. [W, pp. 4, 13]

A universal unrestricted TypeScript-to-WASM compiler is consequently not a prerequisite for the entire idea. Endoforge can admit through different compatible routes and converge at a controlled artifact contract. It can also introduce a smaller computational representation later for recurring pure operations, without forcing every imported application into that representation.

## The narrow end is a contract, not a file extension

The common product is more than executable bytes. It includes input and output schemas, state requirements, declared capabilities, dependency identity, build identity, applicability, provenance, and evidence. Language-independent interface systems such as WIT and the Component Model are relevant precedents named in the discussion, but agreeing on how values cross a boundary does not establish behavioral equivalence. [W, p. 20; R03]

A parser implemented in Python and a replacement implemented in Rust must agree on the required treatment of malformed records, ordering, Unicode, overflow, missing values, errors, and resource limits. A faster implementation that silently changes those rules is a different behavior, not a free optimization.

## Route by total feedback cost

The cheapest route is the one that obtains trustworthy feedback economically for this task. Compilation time alone is insufficient. Preparation, checking, loading, boundary transfers, execution, evaluation, and expected number of uses all matter. A slower-to-build direct implementation may repay its cost during extensive testing; a cheap runtime-backed implementation may be better for a one-shot candidate. [C; R03, R06]

Where a known compiled primitive already solves most of the work, the generator should be able to compose it rather than recreate it. Parameters can often be varied without any new compilation. Existing useful code should not be rejected merely because another language would make a more polished final artifact.

### Keep behavior stable while implementation matures

Promotion and refinement are distinct. A capability can be accepted in its initial implementation and remain useful while a better implementation is constructed. A rewrite gets a new artifact identity, comparison against the actual contract, new qualification, and controlled activation. It does not inherit correctness merely by resembling the previous source. [C; A, sheets 14, 29]

The initial implementation is valuable evidence, but not the sole oracle: it may contain bugs that should not be translated faithfully. State migration belongs at explicit boundaries, not through an assumed transfer of one language's live heap into another's. Some keepers will never justify a rewrite at all.

## 5. One request through the entire factory

The following is a proposed end-to-end demonstration, not an account of a completed experiment. It connects the complete idea around one workpiece instead of treating each subsystem as a separate project. The scenario extends the comparison-workbench demonstration in the original white paper. [W, p. 22; C]

### The request

A user has two datasets and asks for a workbench that compares records, allows grouping and unit changes, explains discrepancies, and exposes the source of each displayed result. The surrounding application already owns the datasets and the user's identity. It provides an admitted snapshot, a small presentation SDK, allowed operations, and a versioned task contract.

Before generating code, Endoforge checks whether the request can be satisfied by composing existing capabilities. A known parser, unit system, table, and filter may already cover most of it. The missing behavior might be only a join rule and a task-specific discrepancy classifier. The system should create that missing behavior, not rebuild an entire UI framework.

If the sources fail to establish which pressure basis or unit interpretation is intended, another compilation will not supply the missing fact. The host must obtain a clarification or represent the uncertainty explicitly. Required semantics become part of the contract before the evaluator treats an answer as correct.

### Admission and the first candidate

The intake router selects an economical supported route. It could accept an existing Python routine through a compatible prepared adapter, use the initial checked TypeScript route, or generate a narrow direct-WASM kernel. This scenario does not assume all of those routes have already been integrated.

The candidate receives a private overlay over approved material. The first version calls a nonexistent SDK method. The semantic checker rejects it before a full renderer test. Structured feedback identifies the mismatch. The repaired candidate builds successfully, but a behavior test finds that it merges two records with distinct source identities. The second repair changes the join logic, not the acceptance oracle.

The local loop need not create a branch, checkout, container, or deployment for every repair. It changes the workpiece and reuses the prepared machinery. Exact source and build identities connect each result to its implementation.

## Qualification and first use

A candidate clears contract, behavior, resource, source-binding, and UI tests. The independent evaluator supplies the acceptance evidence. The host preserves the executable closure and required receipts, rechecks permissions, and activates the result under an expected revision.

The user can now filter rows, change units, inspect sources, and regroup results through working code. Those interactions do not each require a new model explanation. The capability is a small application, even if it is embedded in an existing application rather than installed as a new product.

## A live revision

The user asks for a sensitivity view. The current workbench remains pinned and usable. Endoforge creates a new candidate with the additional behavior and a compatible state migration. The host tests focus, selections, unsaved inputs, source bindings, and error states before changing versions. A stale event cannot silently target a new action. [A, sheets 12–14]

The point is not to animate the search process. It is to let the application acquire a new capability without making its user experience unstable.

## Measured improvement

Once the useful behavior exists, a bounded campaign examines an observed hotspot. One variant changes a threshold; another introduces an index; a third uses a different algorithm. Cheap parameter changes reuse an executable where possible. Distinct implementations retain separate identities and evidence.

A candidate that improves isolated execution but increases total inference, evaluation, or retained memory beyond the protected limits does not automatically replace the incumbent. Finalists are measured independently, including the conditions under which they will actually run. The accepted result is an improvement to the working capability, not merely a winning microbenchmark.

## A keeper and its next implementation

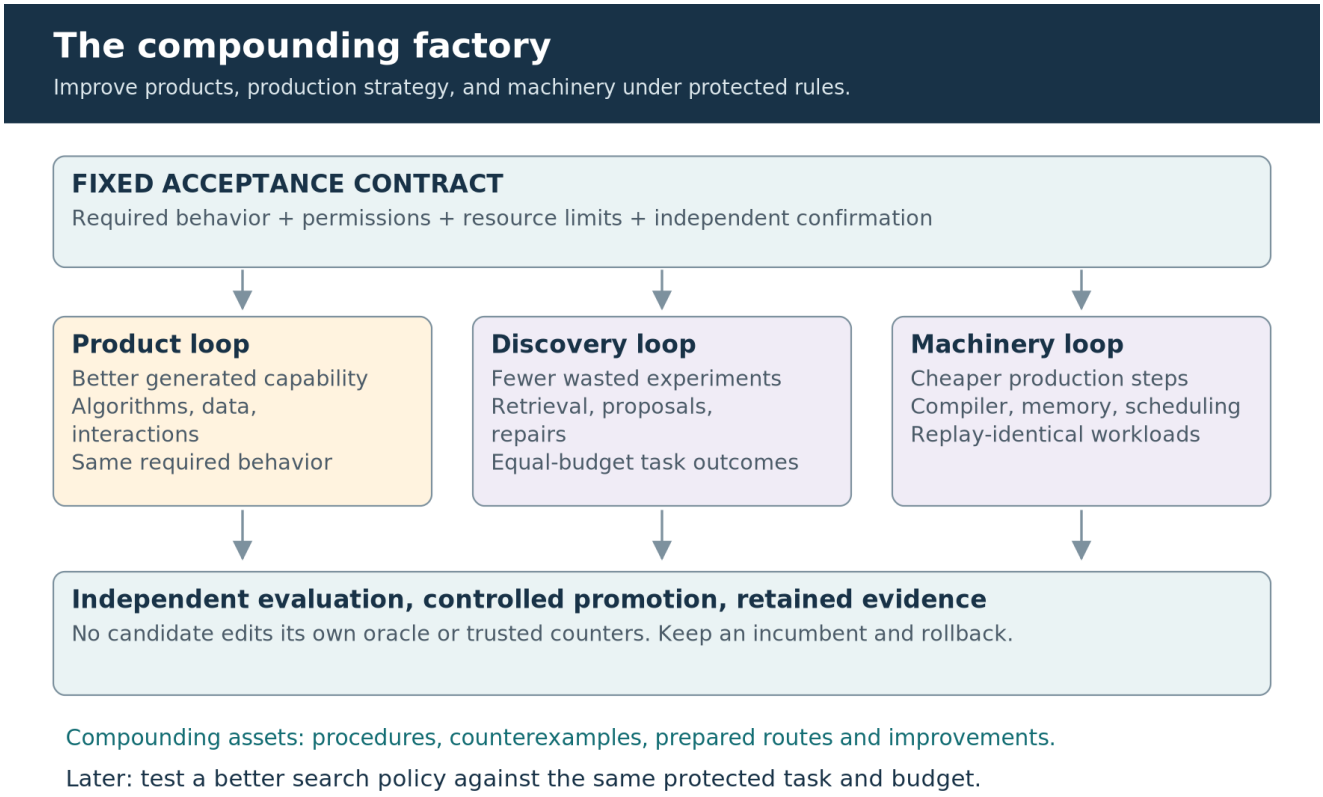
Repeated use makes the parser and classifier worth retaining. Endoforge separates reusable logic from customer-specific input, retains applicability and permissions, and records the tests and known edge cases. Later, a careful Rust rewrite may remove the original language runtime or dependency overhead. The current version stays available while the replacement is qualified.

At restart, the host restores the selected capability and compatible application state from durable records. It does not require the original guest heap to survive. A subsequent task can reuse the parser, contract, or composition pattern; another task can reuse the counterexample that revealed the mistaken join rule. The work has improved both the application and the factory's starting material.

This single journey contains the whole proposition: broad intake, local iteration, independent acceptance, generative software, stable revision, search, selective refinement, durability, and compounding reuse.

# 6. Improvement is a controlled production process

Endoforge has three primary optimization loops. It can improve the software it produces, improve the process that discovers useful software, and improve the workshop machinery itself. A later meta-loop can improve how optimization campaigns allocate their effort. They reinforce one another, but they must be measured separately before their gains are combined. [C; R05]




**Concept map 3. Three primary improvement loops share protected acceptance criteria; the factory may improve proposals and implementations without taking control of its own success definition.**

## Improve the product

The first loop changes an accepted capability's algorithms, layouts, data structures, parsing, state transitions, or other implementation choices. A performance campaign should initially preserve visible behavior. A separate UI campaign can investigate whether an interaction helps the user more, but that requires task and renderer evidence rather than only execution timings. [W, pp. 8–10]

## Improve the production process

The second loop changes proposal strategy, model routing, source context, repair feedback, retrieval, candidate diversity, and stopping rules. Its question is whether the same task quality requires less total

work. A cheaper model response is not a cheaper solution when it creates more failures and retries. A larger swarm is not better when the accepted result is unchanged and total cost rises. [C; W, pp. 7, 9–11]

## Improve the machinery

The third loop changes the compiler bridge, virtual filesystem, memory layout, artifact preparation, caching, allocation, and scheduling. Replay identical source changes and inputs through baseline and challenger to isolate substrate effects. Then run live synthesis under equal budgets to establish whether those effects help the complete task. Both comparisons are needed; model luck must not be mistaken for a runtime improvement. [C; R05, R14]

## What counts as better

Correctness, authority, declared resource limits, source fidelity, and necessary user-state continuity are gates. Among eligible candidates, performance is a vector: latency, CPU, inference usage, peak memory, retained memory, memory occupancy over time, and measured energy where instrumentation exists. CPU time is not an energy measurement. [W, pp. 10–11; A, sheets 08, 11]

A default promotion rule should improve a declared objective while preserving protected metrics within predeclared meaningful tolerances. Some tradeoffs belong on a Pareto frontier as alternative deployment profiles, rather than being concealed inside a single score. Compare against both the immediate incumbent and a fixed reference so a chain of small tolerated regressions cannot accumulate unnoticed. These are campaign-design proposals from the discussion. [C; R05]

All work belongs in the accounting: rejected candidates, repairs, model calls, test generation, evaluation, publishing, rendering, and review. A candidate's speed does not justify removing required tests or changing the meaning of success.

## Make search hard to fool

The evaluator and measurement authority remain outside candidate control. Exploration receives useful feedback; promotion uses controlled confirmation. Repeatedly obtaining scores from the same holdout can adapt the search to that holdout even without direct access to its contents. A release-confirmation set must not be casually treated as untouched after repeated feedback. [W, p. 10; R05, R08]

Randomized paired comparisons, repeated measurements, explicit cache states, controlled workloads, raw samples, and real host accounting reduce the chance of promoting noise. They do not remove every source of uncertainty. An inconclusive difference stays inconclusive. A statistical acceptance rule needs enough observations for the specific objective; subjective usefulness may still require human evidence.

## Spend optimization where it pays

A one-use capsule and a shared compiler path should not receive the same refinement budget. A simple break-even relationship is optimization cost divided by saving per subsequent use, in comparable units. For example, spending 200 CPU-seconds to save 1 millisecond of CPU per execution requires roughly 200,000 executions to repay that CPU expense. Model, review, storage,

and memory costs remain additional. This is illustrative arithmetic, not a measured Endoforge result. [C; R05]

The factory's search policy can itself become a challenger, but it competes on fixed tasks and total budgets. It cannot alter the protected evaluator that decides whether the policy improved. This is controlled recursive engineering, not a claim of unrestricted or inevitable self-improvement.

# 7. The next frontier is removing repeated understanding

RAM residency removes repeated setup and transport. The larger architectural hypothesis is to stop reconstructing the same program meaning for each candidate. Endoforge should aim to make work proportional to the affected region and newly required evidence, not automatically proportional to the entire project. [C; R07]

## Persistent semantic state

A small source edit often changes only a small part of what the system already knows. A future semantic graph could connect source regions, types, interfaces, executable units, dependency identities, test obligations, and established properties. A change would invalidate the facts it can actually affect; unaffected results could be reused under exact assumptions.

The original WASM compiler interface does not establish this capability. It reuses compiled machinery but creates fresh per-run analysis state. Warm tooling and incremental semantic analysis are separate milestones. Endoforge must prove that incremental results match full recomputation, including invalid programs and edits with large transitive effects. [W, pp. 4, 13]

A single edit can legitimately invalidate a large dependency closure. The proposal is not constant-time checking. It is avoiding unnecessary work while preserving sound invalidation and bounded retained state.

## Structured changes instead of source-text round trips

Broad source intake need not make source text the only internal manufacturing format. After admission, compatible candidate families can be represented as structured changes: substitute an approved operation, choose an implementation, bind a parameter, or compose known functions. Human-readable source remains available for review, debugging, import, and export. [C; R07]

This can reduce repeated token output, parsing, and syntax failures. It does not automatically prove semantic correctness. The factory should not force every imported language into one internal representation before that representation has demonstrated value.

## A population is not necessarily a thousand programs

When many candidates share most of their structure, representing them as unrelated programs duplicates preparation. Some searches can use one executable with many parameter blocks. Others can retain a shared immutable program structure and vary only selected regions. Compatible evaluation can prepare common input once while preserving each candidate's private state and observation identity. [C; R07]

The discussion identified e-graphs and equality saturation as relevant research for representing justified equivalent alternatives. An equivalence must come from valid rules under the chosen semantics, not from two programs merely passing a few examples. Different hypotheses remain different until the required evidence supports a stronger relationship. [R07; R18, research references]

Exploration and qualification must also remain distinct. Sharing preparation can make comparison cheaper, but finalists need standalone measurement. A program must not win because it silently relied on another candidate performing work that will be absent after deployment.

### Coalesce dependencies instead of multiplying them

The Mononoke derivation interface provides a concrete warning: its default batch path is sequential in topological order because naive concurrent recursion can cause repeated derivation work. Endoforge should let compatible callers join one in-flight derivation of identical inputs rather than independently reconstructing it. [M, section 3]

An identity match is not permission. Coalescing must account for configuration, source restrictions, trust scope, and actual dependency identity. The objective is shared computation under valid conditions, not indiscriminate sharing of mutable state.

### Specialize machinery for a campaign

A campaign often fixes most of the problem: an SDK, input schema, family of mutations, evaluator, and accepted output shape. A longer-term direction is to prepare specialized machinery around those constants. Partial evaluation and campaign-specific compiler/evaluator paths could leave only the varying work inside each experiment. [C; R07–R08]

This is a significant extension of the original idea. The factory would produce not only better workpieces but also better fixtures and processing paths for the next batch. The cost of creating and qualifying that specialization must be amortized; it is not automatically economical for a one-off task.

### Preserve useful failure

A rejected candidate can leave a counterexample, a known incompatibility, or a constraint that narrows future search. The model can propose a strategy and a bounded variation space, while deterministic machinery explores that space. Reasoning returns when the evidence calls for a genuinely different approach, rather than narrating every local trial. [C; R07–R08]

The objective is more information gained per unit of work. Faster exhaustive search is not the long-term answer to a combinatorial space.

## 8. The memory and trust architecture must support the idea

The intended workcell keeps repeated compilation, execution, local evaluation, and repair in RAM inside an isolate. That is a requirement on the computational path, not a claim that all bytes share one

unrestricted heap or that an isolate provides every enforcement mechanism automatically. [C; W, pp. 4–6]

## Three lifetimes

Durable state holds task identity, admitted inputs, retained candidates, accepted artifacts, evidence, and application state that must survive restart. Warm state holds reusable engine machinery, compatible compiler modules, approved dependency bases, and bounded caches. Attempt state holds private files, compiler graphs, guest memories, scratch data, and transient outputs. [A, sheet 07]

The implementation should reuse the machinery while releasing the workpiece's mutable state. Pending logical work does not need a live environment. The loss of a cache should be a performance event, not a truth event. A live promise is not a durable continuation. [W, p. 6]

## Locality has more than one level

Keeping the computation in one isolate can avoid service and process handoffs, but it does not eliminate UTF-8/UTF-16 conversions, object construction, WASM-memory copies, host buffers, or repeated scans. A source string, a JavaScript object, a WASM linear-memory region, and a GPU allocation are not interchangeable merely because they reside on the same machine. [A, sheet 08; R06, R09]

The practical target is to load a batch once, carry out compatible operations in its resident representation, and return compact evidence. Host-managed handles can expose narrowly authorized operations on buffers without handing candidates arbitrary pointers into trusted state. Whether a particular boundary can be zero-copy is an implementation question to test, not an architecture-label guarantee.

## More than one boundary exists

The outer workcell contains trusted machinery and an inner candidate boundary. Candidate code must not run through an unrestricted evaluation call in the harness that owns policy and measurements. A nested WASM guest, a separately scoped V8 candidate, and a native sandbox are distinct topologies with different locality and enforcement properties. [A, sheets 03–04]

The strongest Endoforge route preserves the same-isolate inner loop. A child-isolate route may be a useful comparator or host-specific alternative, but it is not relabeled as the same topology. Native compatibility paths remain explicit. Inability to fit a route should guide engineering priorities rather than silently relocate every iteration through an external build service.

## Limits must be enforced where they work

A timeout on the same blocked event loop cannot stop a synchronous infinite loop. A resource boundary must cover the actual execution unit and its amplification through host calls. Depending on the host, enforcement may require supported metering, an external watchdog, or termination of an owned process cohort. Shared-process isolates can share process failure; the durable controller must survive and reconcile their attempts. [W, pp. 15, 21; A, sheets 18, 21–22]

Memory accounting includes compiler state, generated code, guest memory, object graphs, stacks, bridges, and native allocations. Binary download size is not peak RAM. Compiled-module sharing across separate isolates is runtime-dependent, not a guaranteed single physical copy across the factory. [W, pp. 6, 11, 13]

### Recovery without invented success

Outputs must exist before accepted references point to them. Activation is a guarded revision change, not replacement midway through compilation. A failure can leave ambiguous execution or unreferenced objects. A leftover result is not proof of canonical completion, and cancellation does not undo effects that have already occurred. [W, pp. 19, 21; A, sheets 21–22]

These rules do not oppose high throughput. They define what an accepted product means, so the factory can optimize around a stable contract instead of generating expensive uncertainty.

## 9. Retained software is the compounding asset

The long-term library is more important than the number of disposable candidates. A useful result can become a versioned capability with interfaces, applicability, tests, known failures, provenance, and one or more implementations. Later tasks can retrieve it, compose it, specialize it, or sponsor refinement. [C; A, sheet 29]

This is explicit procedural memory, outside the model's weights. It stores working procedures rather than only conversational explanations. A successful task can make the next task cheaper even without changing the underlying model. That is an architectural opportunity, not a demonstrated learning curve for Endoforge. [C; R04]

### Keep the capability separate from its implementations

A capability contract describes what an operation means. An artifact identity names exact executable bytes. A qualification record states which behavior and environment were tested. These must not collapse into one record. The same capability may have a low-startup implementation, a direct-WASM high-throughput implementation, and a bounded fallback, each with its own evidence and resource profile. [C; W, p. 20]

A source-language rewrite is therefore only one possible refinement. Better algorithms, data structures, batching, layout, dependencies, or host placement may matter more. The factory should reward the measured improvement, not the language label.

### Do not penalize an idea forever for its first language

A candidate can demonstrate a useful algorithm while having a slow initial implementation. The search should distinguish current operational value from bounded refinement potential. Evidence of better scaling or useful behavior can justify another experiment; it does not entitle the candidate to promotion before the improvement exists. [C; R03]

This is why the funnel and the finishing stage belong together. Broad intake lets useful ideas enter cheaply. Controlled refinement lets them mature without requiring every candidate to start in the ideal production language.

### Reuse must preserve context

Generated source can embed confidential values. A reusable template should separate data-free logic from scoped input where possible, but promotion does not itself remove inherited restrictions. Every reuse needs current permission, source context, and compatibility checks. Previous acceptance is not blanket authority. [W, p. 20; A, sheets 20, 29]

Likewise, performance evidence has scope. A capability that is efficient for one input distribution or machine may not be efficient for another. The library should retain that applicability rather than presenting a universal ranking.

### The mature factory may generate less code

The far-end hypothesis is a system that already contains much of the structure needed for familiar tasks. A request becomes retrieval, composition, binding, a small specialization, and targeted qualification. Genuine novelty triggers deeper reasoning and search. The desired accumulation is less repeated work, not an ever-growing landfill of unreviewed code. [C; R07–R08]

## 10. The ledger belongs inside the factory, not at its center

The Scion discussion focused attention on source-control infrastructure as a potential bottleneck when machine-generated change volume rises. The subsequent Mononoke study supplied concrete mechanisms that fit Endoforge's candidate lifecycle. Their role is supporting production; Endoforge is not defined as a Git replacement. [C; M, sections 1–10]

### What carries over from Mononoke

Bonsai records ordered parents, metadata, and changed-path mappings while file contents live separately. The derivation framework separates canonical records from representations that can be generated on demand. Ephemeral bubbles provide temporary storage overlays. Sharded maps support bounded loading. Conditional publication protects mutable references. [M, sections 1–7]

The source study's recommendation is a small Endoforge-native ledger, not embedding the whole Mononoke server or dependency closure into WASM. The prior shell clone failed; 19 source files or specified ranges were studied through the connector at commit `79071c2a3ea20c36ed71332608535e5801101bba`. No upstream build or tests were run. The preserved archive includes the study and checkout scripts, not a completed clone. [M, scope]

### Separate history from computation identity

A candidate/provenance identity records the proposal and its lineage. A resolved source identity names the effective source and dependency state. A build key names all build-affecting inputs. An artifact identity names exact output bytes. An evaluation attempt names an observation. An activation record names the authorized transition to use. [M, section 1]

Two proposals can therefore share a build without becoming the same historical event. Conversely, the same executable can have many observations under different inputs or machines. A hash names bytes; it does not grant permission or prove task correctness.

### Do not manufacture every representation

An admitted candidate need not immediately produce a release archive, Git tree, blame index, polished explanation, or optimized final binary. The factory should request the dependency closure required by the next step. Required correctness gates stay in that closure before activation. Optional packaging and historical indexes can wait. [M, section 3]

A changeset record is not an incremental compiler graph, however. Cheap source branching does not automatically reduce semantic analysis. The ledger and compiler state need an explicit connection.

### Derived results and observations are not the same

A deterministic representation can be regenerated from complete pinned inputs. A physical timing sample, external response, model output, or reviewer judgment is an observation of an event. Repeating it produces another observation, not the original event. Evidence therefore belongs in a journal with its scope preserved, rather than in a casually disposable cache. [M, section 3]

Publication follows object closure and accepted evidence. A conditional update that loses its race does not activate the candidate. Lost acknowledgments require reconciliation against the authoritative activation record; a failed compare-and-swap alone cannot establish that nothing happened. [M, section 7]

The whole system benefits when the ledger lets many alternatives exist cheaply, preserves accountability, and avoids duplicate work. It becomes a bottleneck if every internal experiment must pass through a full human-oriented repository workflow or durable publication of every scratch object.

## 11. Reason upward from physics, not downward from today's workflow

The physical-limits discussion broadened the design beyond making an ordinary build loop faster. Physics does not require a checkout, a new compiler process, a fresh source parse for a parameter change, or a model turn for every deterministic operation. The larger opportunity is to remove work that the task never required. The following preserves that reasoning as theory and architectural inference, not measured Endoforge performance. [C; R08]

### There is no universal maximum programs per second

A meaningful physical limit needs a defined task, available information, correctness requirement, machine size, energy, temperature, error tolerance, and required external interactions. A three-operation transformation and a complex simulation are not interchangeable production units.

A useful simplified lower-envelope model is:

```
T >= max(W/F, Q/B, T_critical_path, T_required_external_interaction)
```

Here W is computational work, F available computational throughput, Q information traffic across a relevant boundary, and B effective bandwidth. Serial dependencies and necessary external waits also impose floors. This is not a complete timing predictor, because phases may not overlap and shared resources can add contention. Its value is showing that Endoforge can change W, Q, and the dependency graph instead of merely buying a higher F. [R08; R18, research references]

## Distance and locality

Using the exact vacuum speed of light, ideal one-way propagation is about 33 picoseconds across a centimeter, 3.3 nanoseconds across a meter, and 3.3 milliseconds across 1,000 kilometers. These are calculated physical floors, not practical software latencies. Real systems add transport, queuing, switching, and processing. [R08; R18, EXT-PHY-01]

The design consequence is that repeated dependence on distant state can dominate even an extremely fast local compiler. Isolate locality is the first level. Cache, DRAM, chip interconnect, accelerator placement, machine boundaries, and model-service distance are further levels. The goal is a short and infrequent necessary communication path.

## Movement can dominate the calculation

The discussion used an illustrative workload with 1.28 GB of DRAM traffic per candidate. At an assumed effective 100 GB/s, the transfer term alone is 12.8 milliseconds. If a change-aware path needs 256 kB instead, the corresponding term is 2.56 microseconds: a 5,000-fold difference in that traffic term, not a forecast of a 5,000-fold application speedup. [R08]

This example connects directly to persistent semantic state, shared candidates, compact representations, batch processing, and reuse of prepared data. A more efficient representation can remove far more work than a modestly faster implementation of the same repeated full scan. Sometimes recomputing a small value can also be cheaper than retaining and retrieving it; memory residency is not automatically optimal in every case. [C; R08]

## Thermodynamics and state transition

For the standard isothermal erasure of an unbiased classical bit, the Landauer expression is `E_erase >= k_B T ln(2)`. At 300 K this is approximately `2.87e-21 J` per erased bit. It is not energy per instruction, token, compiler check, or software candidate. Reversible computation and finite-time erasure were also considered in the conversation: avoiding logical erasure at every step does not make practical machinery lossless, and minimum dissipation is not generally the same operating point as maximum throughput. [R08; R18, EXT-PHY-02–04]

The Margolus–Levitin bound discussed there concerns evolution to an orthogonal quantum state, with `tau >= pi hbar / (2E)` under its assumptions. E is energy above the ground state, not machine wattage. Lloyd's one-kilogram, one-liter ultimate-computer thought experiment places an extremely distant theoretical envelope near `5.4e50` elementary operations per second under extreme mass-

energy conditions. These are not consumer-hardware roadmaps or Endoforge specifications. [R08; R18, EXT-PHY-05–06]

Their role in the idea is to separate fundamental limits from today's architectural conventions. Endoforge can pursue enormous reductions in redundancy long before approaching those physical extremes.

### Search and information are limits too

One hundred independent binary choices create `2^100` combinations. At an assumed trillion evaluations per second, exhaustive enumeration would take about 40 billion years. This illustrative calculation explains why more candidates alone is not the destination. Better representations, useful priors, parameter families, valid equivalences, and discriminating counterexamples matter. [R08]

When the intended behavior is underdetermined by the available information, computation cannot manufacture the missing observation. The efficient operation may be a clarification or an authoritative lookup. Verification of unrestricted programs also has fundamental limits, while bounded domains and explicit contracts still permit strong checking. The factory should seek useful restricted proof and validation systems without claiming that passing tests establishes every possible behavior. [R08]

### The direction toward the limit

The practical sequence is to remove repeated setup, then unaffected reanalysis, then duplicate candidate work, then general-purpose overhead that can be economically specialized, and then avoidable movement across hardware boundaries. Proof or validation can preserve some evidence under explicit assumptions; physical performance still needs observations.

The far-end ambition is **only the new behavior, only the information that must move, and only the evidence that the change invalidates**. That is a research direction, not a promise that all necessary work can be reduced to a constant-time lookup.

## 12. Unified-memory systems are a physical fit, not an automatic win

The user connected the RAM-resident factory to Apple M-series and AMD Strix-class unified-memory SoCs. The attraction is alignment between software locality and hardware locality: reasoning, compilation, evaluation, data, and candidate state can be placed close together on an integrated machine. The prior discussion identified Apple MLX/Metal and AMD GPU tooling as implementation directions. Exact model capacities, bandwidth, APIs, and compatibility remain host-qualification tasks; this edition does not assert new current hardware specifications. [C; R09, R18]

### Proposed placement

CPU resources handle compilation, semantic work, scheduling, ordinary WASM execution, and branch-heavy tests. An integrated GPU can handle local inference and sufficiently large compatible numerical batches. Shared host-managed allocations hold admitted reusable data where the software stack supports it. Private allocations hold candidate state. Persistent storage holds accepted artifacts,

evidence, and checkpoints rather than mandatory scratch-file round trips between every operation. [C; R09]

The reasoning model should be shared production machinery, not a separate copy loaded for every agent. Its service can be physically local while remaining outside the candidate's trust domain. Likewise, one physical memory pool does not require one unrestricted address space.

### Shared memory also means shared demand

A simplified budget is:

```
B_inference + B_compiler + B_evaluator + B_other <= B_sustainable
```

The terms describe concurrent demand on shared memory bandwidth, not separate guaranteed bandwidth allocations. Overlapping inference and compilation can improve throughput, or contention can make the combined result worse. The factory must measure both cases. [C; R09]

Memory capacity determines what fits, not how fast the complete loop works. Model weights, context state, compiler graphs, guest memory, GPU buffers, caches, and the host all compete for capacity. Thermal limits and sustained operating conditions also belong in the experiment. A cold burst is not a sustained-production guarantee.

### Zero-copy is an interface property too

Shared physical memory can avoid some transfers between separate device memories, but allocations still need compatible layout, ownership, synchronization, and lifetime rules. Ordinary WASM does not automatically execute on a GPU. Accelerated kernels require an appropriate backend and governed host interface. [R09]

An optimized discrete-GPU system can also keep its data resident rather than uploading it repeatedly. It is therefore an important comparator where relevant. Endoforge should measure useful output and energy under matched workloads rather than assume a packaging label decides the outcome.

### The hardware–software opportunity

Reducing compiler traffic may free bandwidth for inference; batching evaluations may reduce bridge overhead; better scheduling may overlap cache-local work with bandwidth-heavy work. These gains can reinforce each other. They are examples of optimizing the production process across resources rather than tuning each machine in isolation.

The proposed first experiment measures compilation/evaluation alone, inference alone, and the combined loop, followed by concurrency and memory sweeps. The metric remains accepted output under the same quality contract per unit of time and measured total energy. No Endoforge energy result has been established by the supplied studies.

## 13. The applications are expressions of the same substrate

Generative UI, agent tools, analytical workbenches, internal workflows, educational instruments, simulations, and personal applications are possible products of the same mechanism. The original

white paper names this broader application range. The purpose is not to force every domain into one UI, but to provide a common ability to create and improve bounded software where it is useful. [W, p. 23]

## Generative applications

A host supplies identity, stable components, data contracts, and effect policy. Endoforge supplies task-specific behavior and presentation definitions. The user works in a stable instrument while revisions are manufactured separately. The active version changes only when the host can preserve state and authority. [W, p. 8; A, sheets 12–14]

## Agentic coding and tool creation

An agent supplies intent, source, or a patch. The factory performs deterministic checking, execution, and local search, then returns compact evidence. It can manufacture a reusable tool rather than requiring the model to reason through the same manipulation at every use. This is not a claim that models are unnecessary; they remain valuable for choosing approaches, interpreting evidence, and resolving new intent. [W, p. 7]

## Local, client, site, and cloud hosts

The portable unit is a capability and checkpoint contract. A browser, local workstation, site server, and managed cloud can implement that contract under different APIs and resource ceilings. Code, artifacts, and checkpoints can move; arbitrary live heaps, host credentials, native objects, and GPU allocations are not assumed portable. [W, p. 21; A, sheet 16]

A managed cloud route can supply elastic capacity and platform enforcement. A local route can keep compatible work near private data and operate offline where dependencies permit. A browser route can keep an interactive workcell separate from the UI thread. These are deployment expressions of the broader design, not interchangeable claims about exact isolation or performance.

Cloudflare Code Mode, Dynamic Workers, and workerd were relevant adjacent systems in the discussion. Their presence means the project cannot rely on the claim that nobody has considered runtime-generated tools. Endoforge's differentiator must be the complete integrated process and demonstrated economics: local creation, meaningful qualification, stable use, refinement, and compounding reuse. Current platform behavior must be checked on the exact host before implementation. [W, pp. 14–15; R18]

## Field OS stays an application

Field OS supplies an applied setting and recorded subset: source-bound tools, admission, controlled guest execution, custody, and recovery. Endoforge is the general runtime factory beneath that use case. A useful implementation in Field OS can inform the substrate without making refrigeration, its UI, or its existing native stack the definition of the platform. [W, pp. 16–19; A, sheet 27]

# 14. What is supported now, and what remains to be demonstrated

The supplied material establishes useful pieces, not the complete factory. Keeping that distinction clear makes the programme more actionable: each unproven connection becomes a specific integration or experiment instead of an assumed capability. [W, pp. 16–23]

## Recorded applied evidence

The white paper reports warm guest-adapter medians of 18.64 ms with shared VFS and 18.65 ms with fresh VFS at its stated 100% CPU quota for a particular six-case synthetic fixture. A preloaded native function was much cheaper at 0.131 ms. The fresh/shared difference is not a statistically established advantage. These numbers support investigation of prepared machinery with fresh state; they are not an Endoforge end-to-end speedup. [W, p. 17]

The same source records named containment, return, and recovery checks, and preserves the failed broader test scope. It also records a failed local 27B CPU-only model request. Those results establish scoped behavior and cleanup, not broad model usefulness, physical power-loss survival, multi-tenant security, or a working generative-UI swarm platform. [W, pp. 18–19]

## Compiler and source-study evidence

The original compiler figures are upstream native reports, not in-isolate WASM timings. The WASM interface demonstrates an embedding surface in the cited documentation, but a complete target-host integration and incremental semantic service remain separate work. The Mononoke study is source inspection, not a deployment, WASM port, or benchmark. [W, pp. 12–13; M, scope]

## The important unknowns

The programme must establish that the selected frontend really operates in the intended isolate, that required checking and emission behave correctly, that guest execution is bounded, that state is reclaimed, and that a generated capability helps an actual user task. It must then establish whether candidate search improves outcomes at an equal budget, whether refinement repays itself, and whether shared representations reduce total cost without invalidating evidence. [W, p. 22; R13–R15]

The hardest bottleneck may migrate as the system improves. First it may be setup, then checking, then data movement, then inference, then evaluation, then the lack of a sufficiently precise task contract. The factory needs measurements that expose this migration instead of continuing to optimize yesterday's bottleneck.

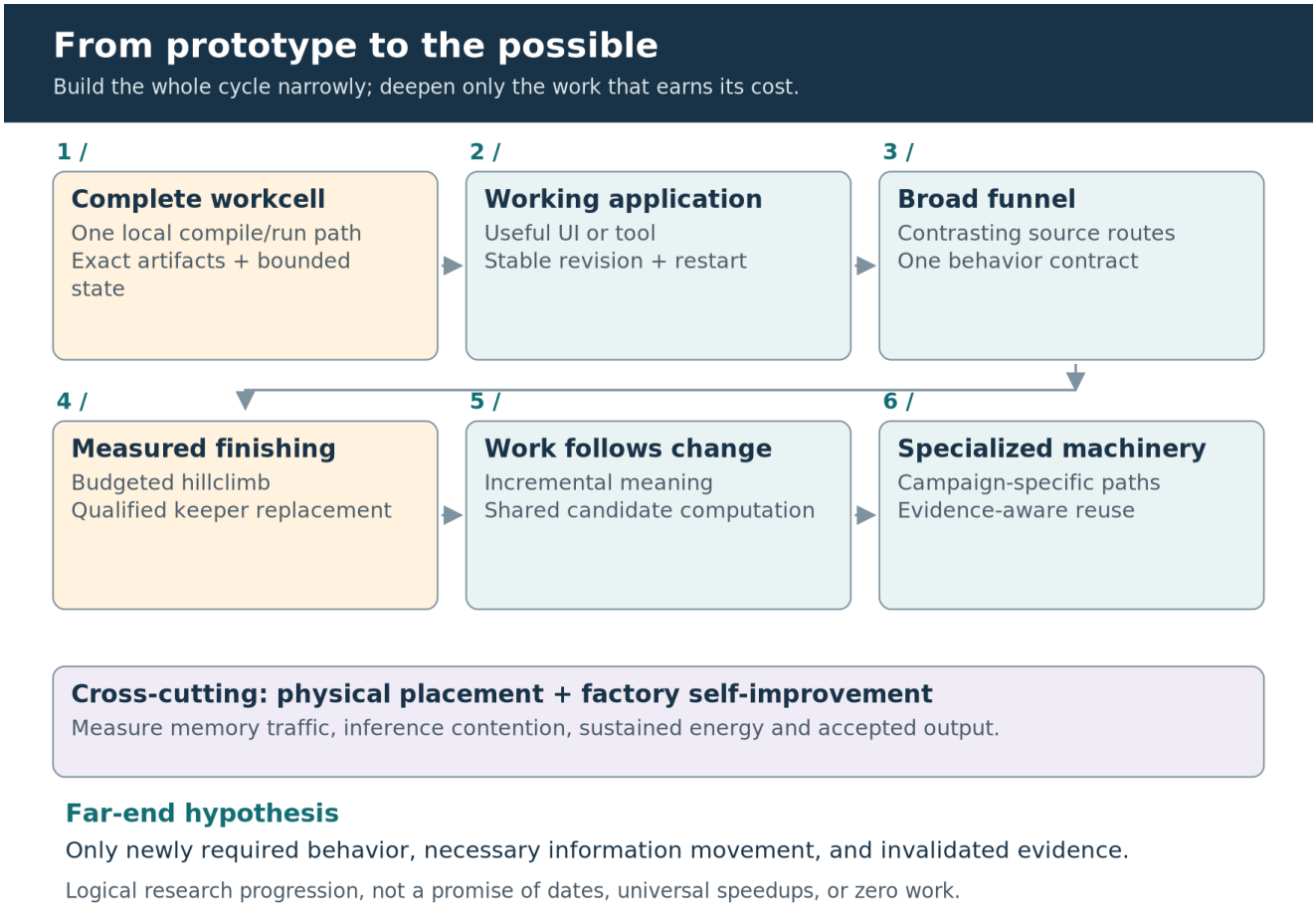
## A claim ladder

A local compiler is an embedding result. A bounded guest is an execution result. A passing generated instrument is a task result. A better campaign under equal cost is an optimization result. Repeated, efficient retained use is a lifetime-economics result. A more effective factory after accepted machinery changes is a recursive-engineering result.

None of those should be substituted for another. The purpose of the complete idea is to connect them, not to multiply unrelated ratios or assume the strongest claim follows from the first successful component.

# 15. From today's components to the possible

The practical route is to build the whole cycle narrowly, then deepen the machinery. It should not spend its first milestones on a universal compiler, a complete Git replacement, or exotic hardware. Those directions matter only where they address a measured constraint on the complete product. [C; W, pp. 22–23]




**Concept map 4. Development path from a complete narrow workcell to broader intake, controlled improvement, incremental shared computation, and hardware-aware specialization. Each stage requires evidence rather than a calendar promise.**

## Establish one complete workcell

Pin a task contract, source inputs, runtime, compiler, SDK, evaluator, and resource profile. Put one supported compile/check/run path inside the workshop. Show malformed input rejection, bounded execution, current permissions, exact artifacts, and actual reclamation. Compare against an already-warm native or declarative baseline, not a fictional cold operating system per candidate. [W, p. 22; A, sheet 28]

Then deliver one useful generated interface with behavior, source inspection, and a state-preserving revision. Demonstrate restart and rollback. The result should be usable, not only successfully compiled.

## Establish the funnel through contrasting routes

Add another meaningfully different supported source route while preserving the capability contract. Prove the required frontend location, dependencies, runtime semantics, and memory envelope. Use the same behavior contract to compare runtime-backed and directly compiled implementations. Where a route cannot yet live in the workcell, record the missing integration rather than disguising an external build as local.

## Establish controlled improvement

Run one measured campaign over a bounded hotspot or retained capsule. Preserve the incumbent, maintain independent acceptance, count complete costs, and validate finalists under held-out conditions. Then refine a selected keeper across implementation choices and demonstrate that the replacement preserves required behavior and user state. This closes the discovery-to-finishing cycle.

## Remove repeated work systematically

Begin with low-risk reductions: smaller generated surfaces, prepared dependencies, artifact reuse, parameterized candidates, batching, fewer conversions, and actual memory reclamation. Proceed to incremental semantic analysis, dependency-keyed coalescing, shared candidate families, and specialized campaign machinery only when each is backed by full-recomputation comparisons and total-resource measurements. [C; R06–R07]

This is the route from faster execution of today's workflow to a different computational workload. The size of the affected region and the information gained become first-class metrics.

## Improve the factory itself

Once campaign controls are dependable, give Endoforge bounded responsibility for its own subsystem improvements. A runtime, compiler, or search-policy challenger is still a candidate under an external acceptance contract. Version and retain the evidence for why a change was selected. Test the factory against both its predecessor and a stable reference so small regressions do not disappear into the lineage.

## Co-design placement with actual hardware

Measure CPU, GPU, memory, inference, and evaluator contention on the intended hosts. Tune scheduling and data layout for complete accepted outcomes. A unified-memory machine is an important route, not an excuse to skip a suitable discrete or cloud comparison. More distant processing-near-memory or reversible techniques remain research horizons, not dependencies for proving the product.

### The mature endpoint

The strongest plausible endpoint discussed is a runtime holding a growing body of reusable computational structure, contracts, and scoped evidence. A new request causes only the necessary new reasoning, changes, execution, and qualification. Familiar work becomes binding and specialization; genuinely novel work receives deeper search. Successful improvements make subsequent production cheaper without weakening what counts as success. [C; R08]

That is not an assertion of unlimited acceleration. Irreducible task work, missing information, measurement uncertainty, communication distance, and physical resource tradeoffs remain. The point is to approach those real constraints by removing conventions and duplication that never belonged to the task.

**Endoforge is a proposal for software that can manufacture more software as part of doing its work—and for that manufacturing process to accumulate capability and improve its own efficiency over time.** The in-isolate RAM-resident workcell is its production core. The broad funnel supplies material. Independent evidence makes the output usable. Refinement and reuse make the economics compound. The whole is the idea.

## Appendix A. Reading the supporting studies as parts of one system

The complete earlier engineering package is preserved under `05_Engineering_Reference`. It is a supporting library, not a replacement for this narrative. Its documents, benchmark programme, work packages, templates, diagrams, original sources, and integrity tools retain their original identities and qualification notes.

| Part of the entire idea | Supporting material in the reference library |
|---|---|
| Founding thesis and factory model | Documents 01–02; W, pp. 1–7; A, sheets 01–09 |
| Broad language funnel and keeper finishing | Document 03; C; W, pp. 4, 13 |
| Working generative applications and retained capability | Document 04; A, sheets 12–14, 29 |
| Three improvement loops and campaign controls | Document 05; W, pp. 9–11 |
| Low-hanging performance reductions | Document 06 |

| Part of the entire idea | Supporting material in the reference library |
|---|---|
| Persistent semantics, shared search, and specialized machinery | Document 07 |
| Physical constraints and far-end hypotheses | Document 08 and illustrative calculations |
| Unified-memory and local reasoning placement | Document 09 and hardware qualification template |
| Scion motivation, Mononoke source study, and experiment ledger | Documents 10–11; original M study and pinned source map |
| Recorded observations versus missing proof | Document 12; W, pp. 12–19 |
| Implementation sequence and matched experiments | Documents 13–14; 32 work packages; 18 campaign records |
| Risks, decisions, terminology, and sources | Documents 15–18; registers; original source PDFs |

The new execution plan in `02_Execution_Plan` connects these streams around a complete vertical slice and explicit gates. The coverage map in `04_Coverage_and_Evidence` traces each major conversation topic into this central document and the retained studies.

# Appendix B. Source and status key

**C — Conversation.** The naming, broad funnel, in-isolate/RAM requirement, real-factory framing, progressive refinement, optimization directions, physical reasoning, and unified-memory direction were developed here. This document is a synthesis, not a verbatim transcript export. The user's selected directions are distinguished from proposed implementations and speculative endpoints.

**W — Original white paper.** *The In-Isolate Software Workshop*, 14 October 2026, version 1.0, 25 pages. Its recorded and upstream-reported values retain their original scope. Included unchanged in `05_Engineering_Reference/05_ORIGINAL_SOURCES`.

**A — Original atlas.** *Diagram Atlas*, 34 pages, with 30 numbered diagram sheets. Sheet 01 is PDF page 3 and sheet 30 is PDF page 32. Original diagrams and individual vector extracts are preserved in the reference library. The four new concept maps summarize the evolved proposal; they are not architecture measurements.

**M — Mononoke study.** *Mononoke mechanisms for Endoforge*, 16 October 2026. Nineteen source files or ranges were reported inspected at `79071c2a3ea20c36ed71332608535e5801101bba`. The prior clone failed; source study succeeded; upstream build and tests were not run. The original study package and source map remain intact.

**R01–R18 — Engineering reference documents.** These are the 18 standalone documents from the previous package, included in both PDF and Markdown. Their source register preserves the earlier external links and time-sensitive claims with revalidation requirements. This synthesis does not newly verify Scion availability, manufacturer specifications, compiler releases, platform limits, or license interpretations.

**Recorded** means reported by the supplied source, not rerun here. **Proposed** means intended architecture or experiments. **Illustrative** means a calculation from stated assumptions. **Speculative** means a longer-term possibility. Document rendering and package-integrity checks do not establish runtime correctness or performance.